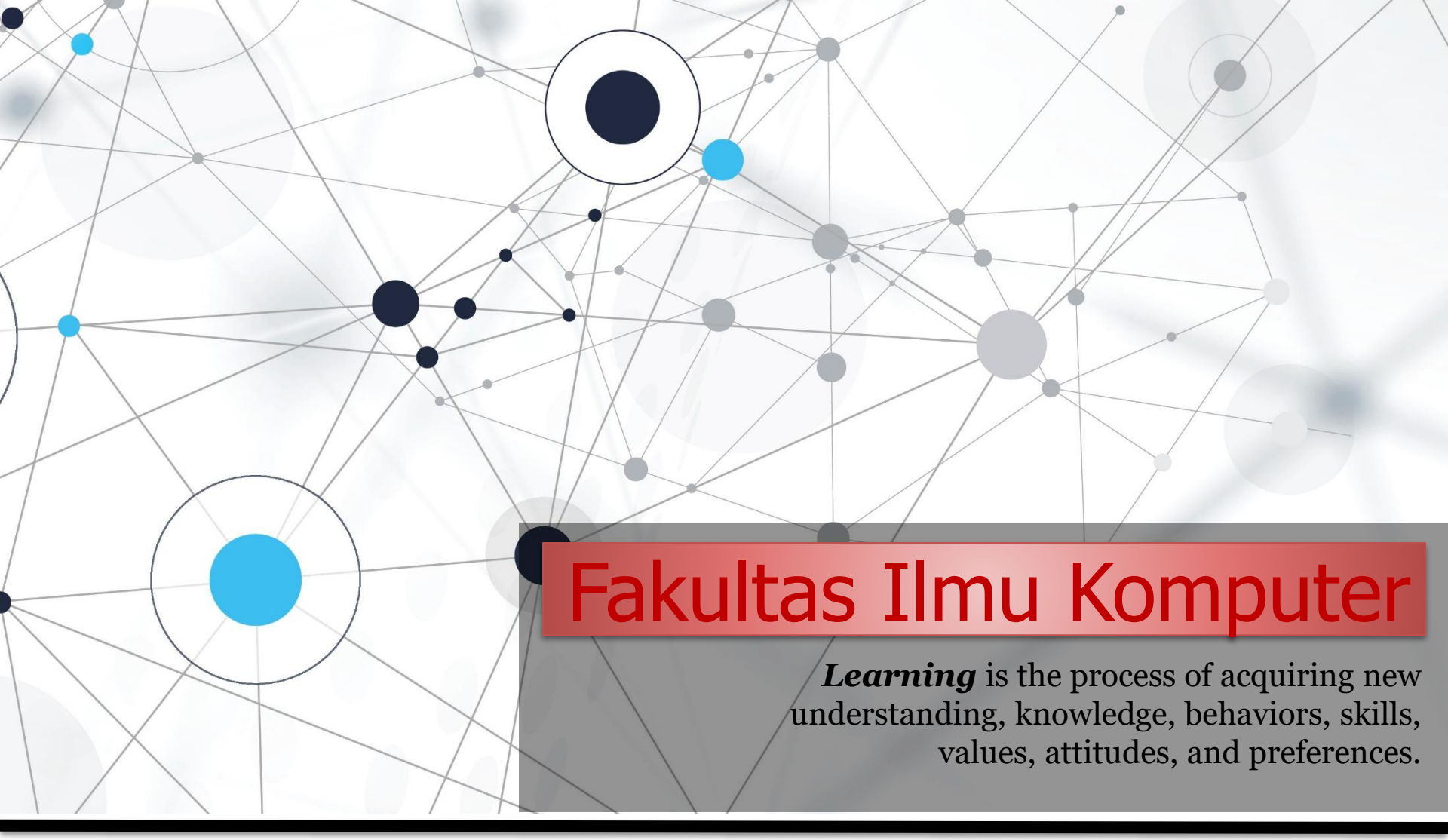


PHP MVC (*model, view, controller*)

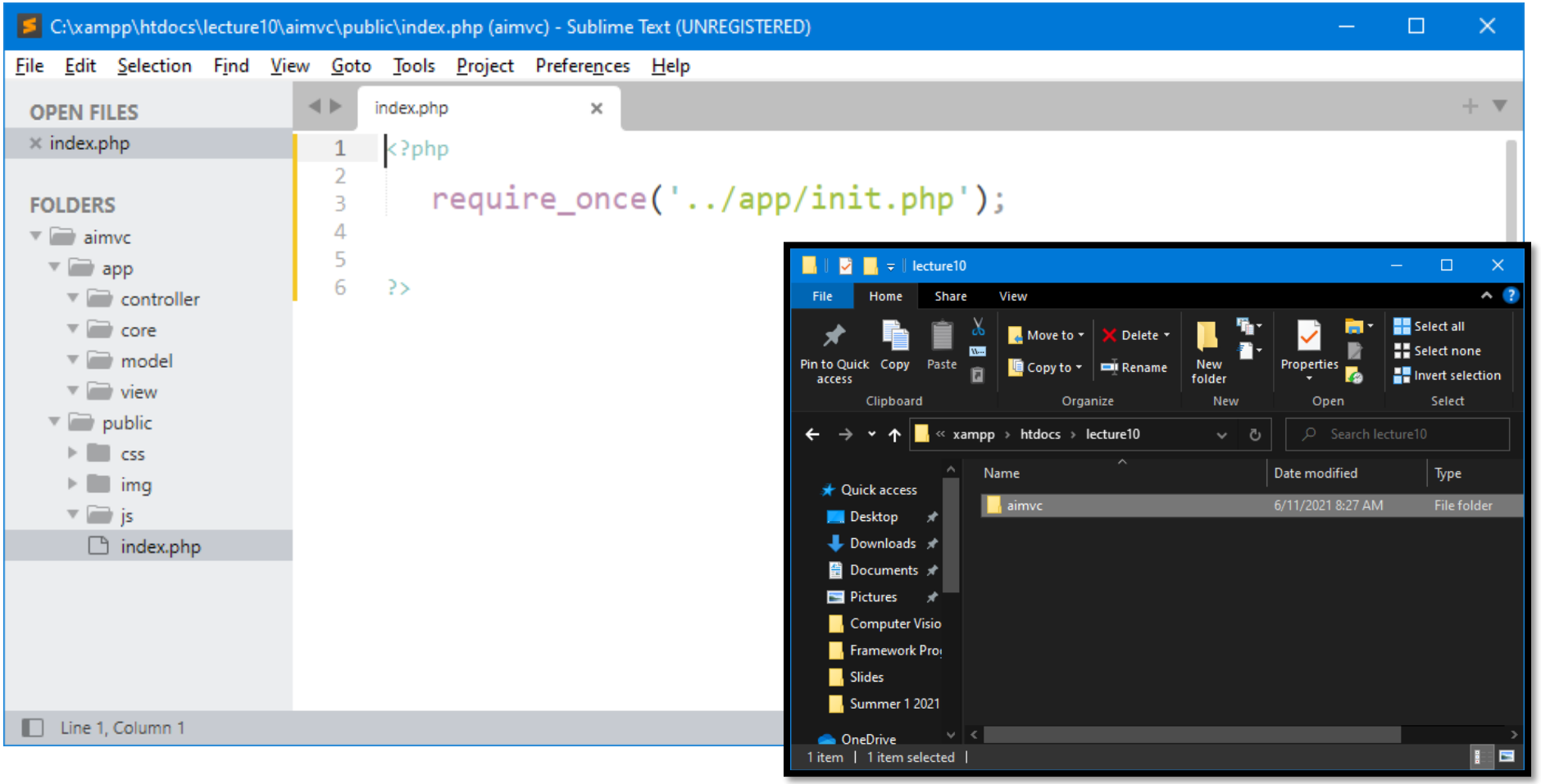
Framework #2 – Structure & URL Routing

MVC File and Directory Structure
URL Parsing with Router (*URL Routing*)

1 - File & Directory Structures

Basic File & Directory Structures

Directory Path: C:/xampp/htdocs/lecture10/aimvc/



Main directory of our new framework:

aimvc ⇨ **A**rtificial **I**ntelligence **M**odel **V**iew **C**ontroller ⇨ PHP Framework

Create Framework Structure

No	Directory/File	Description (*optional*)
1	app	app directory is the web application's ***private directory*** of this new framework.
2	public	***public directory*** is to store all the public static resources like ***html***, ***css***, ***image*** and ***js*** files.
3	core	core directory is to store all files for routing and it will store the framework's core classes.
4	model	model directory is for all app's Model classes.
5	view	view directory is for all app's View classes.
6	controller	controller directory is for all app's Controller classes.
7	css	css directory is for Cascading Style Sheets (CSS) files.
8	img	Img directory is for images files.
9	js	js directory is for JavaScript files.
10	index.php	index.php file is the 'entry-point' file, the front controller to the users.

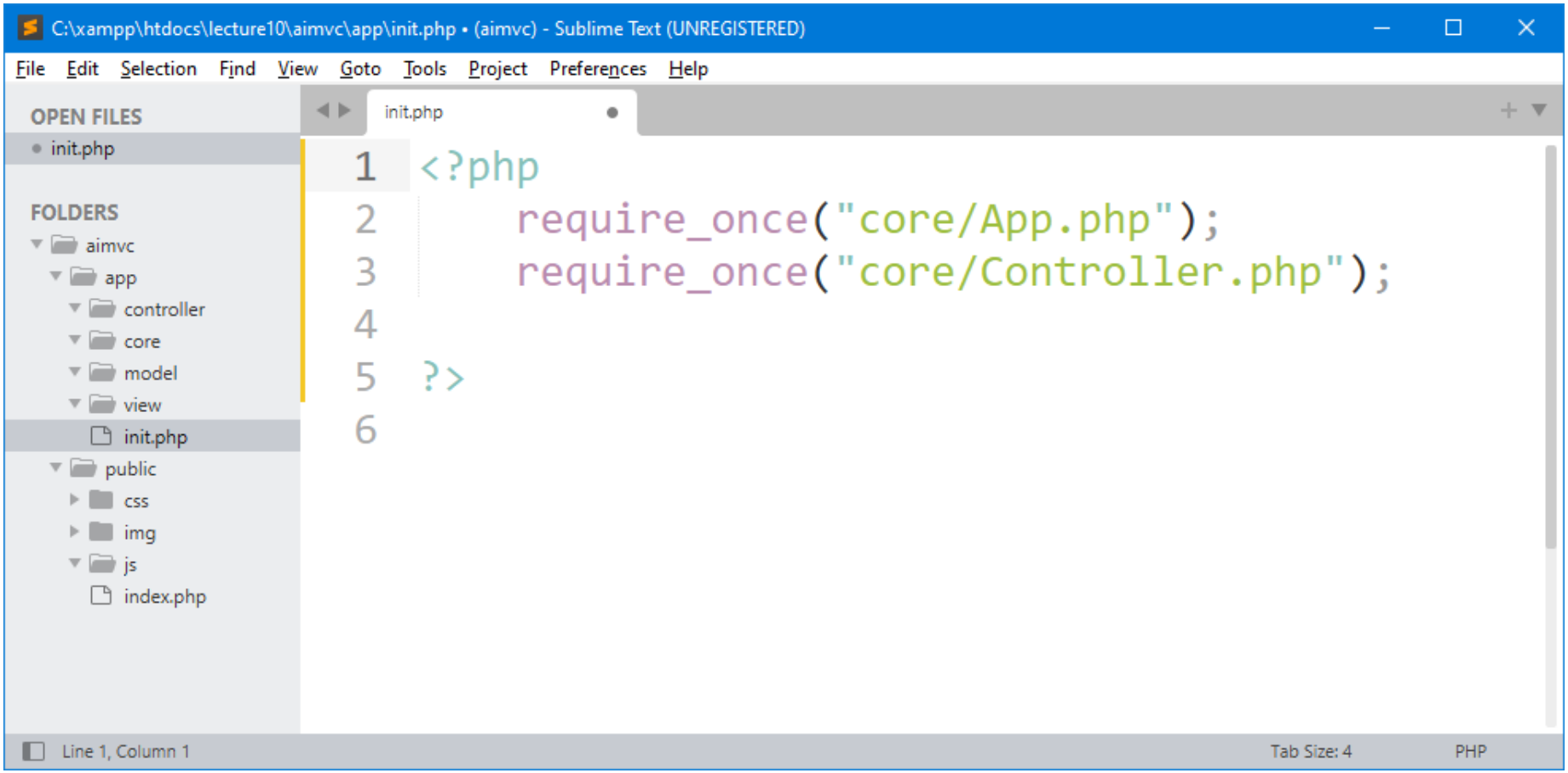
Additional Directories:

template directory stores the template files like *header* file and *footer* file.
config directory stores the app's configuration files.
upload directory is for uploaded files, such as uploaded images.
db directory is database related classes, such as database driver classes.
helper directory is help/assistant functions.
library directory is for class libraries.

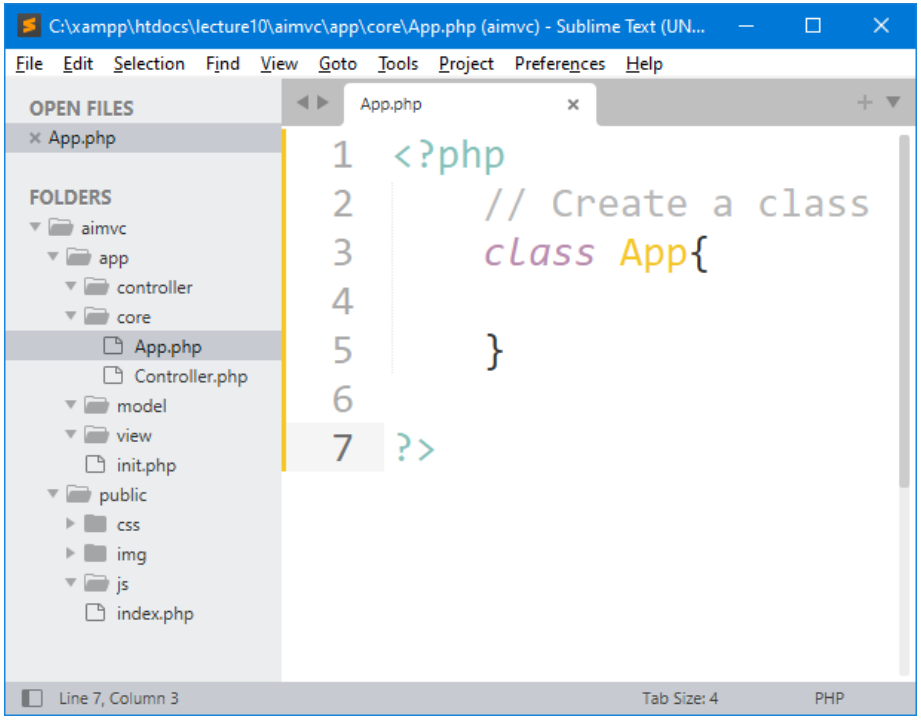
Create The file *init.php*

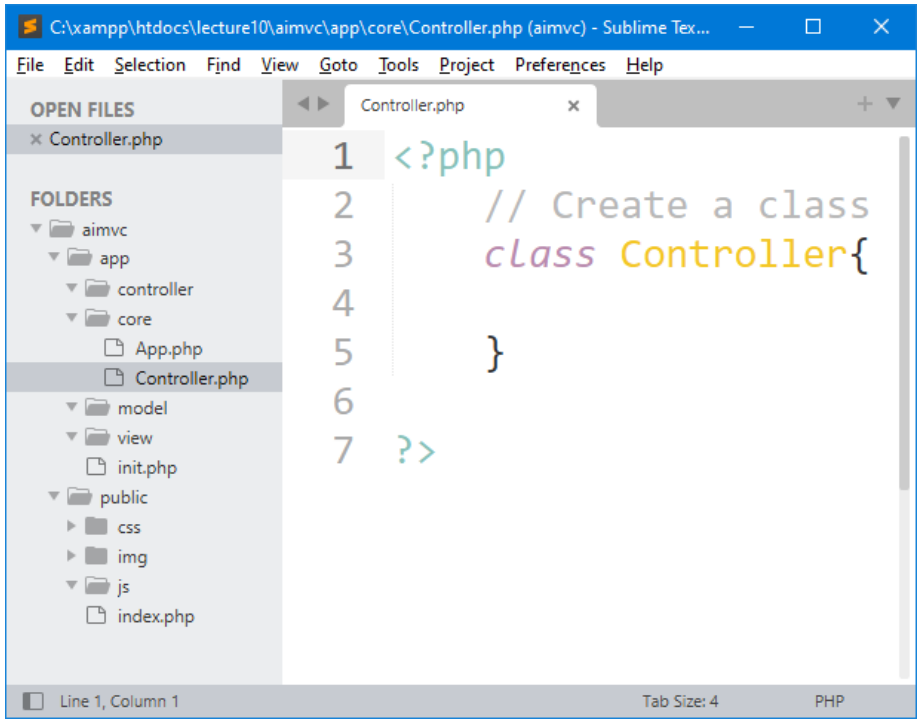
init.php (*initialization*) is an optional file within our Framework file structure and located in *private directory*. This file may contain the initialization of event handlers and connection of additional functions that are common for all websites.

File Path: C:/xampp/htdocs/lecture10/aimvc/app/ (*private directory*)



Create Framework's Core Classes



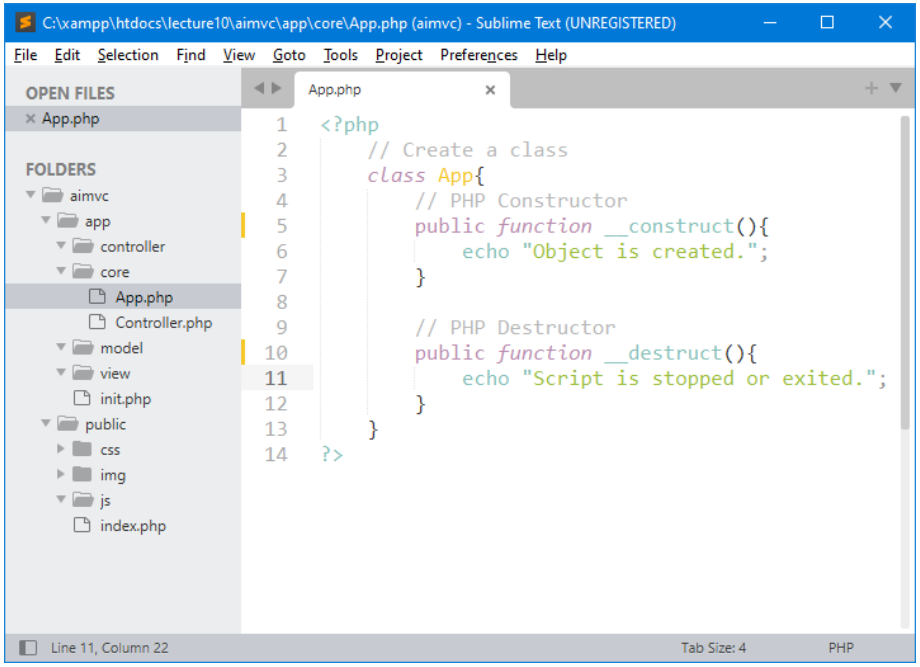


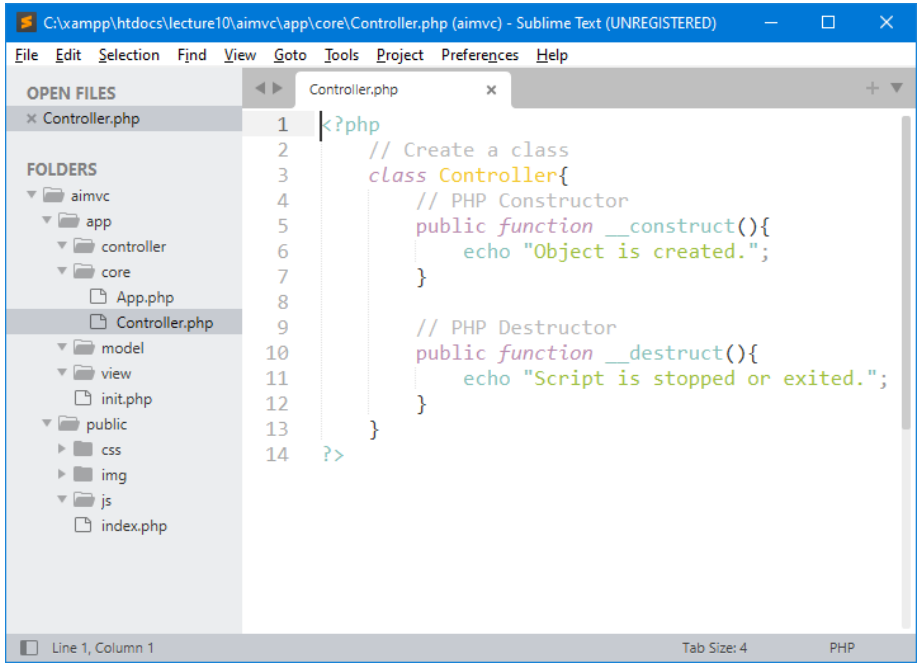
Class App (*App.php*) digunakan sebagai class utama dalam pembentukan aimvc framework. Class Controller (*Controller.php*) digunakan sebagai class yang akan mengatur semua fungsi-fungsi yang akan diletakan di dalam *controller directory*.

Note:
First letter as a capital letter (*uppercase letter*) normally means a php class.

Constructors and Destructors

Tambahkan fungsi **___construct** dan **___desctruct** di Class App (***App.php***) dan Class Controller (***Controller.php***), ini berfungsi agar supaya pada saat proses instansiasi atau proses pembuatan instance/objek dari class App (***App.php***) dan class Controller (***Controller.php***) selesai, maka sudah bisa langsung di *display* contoh output-nya.



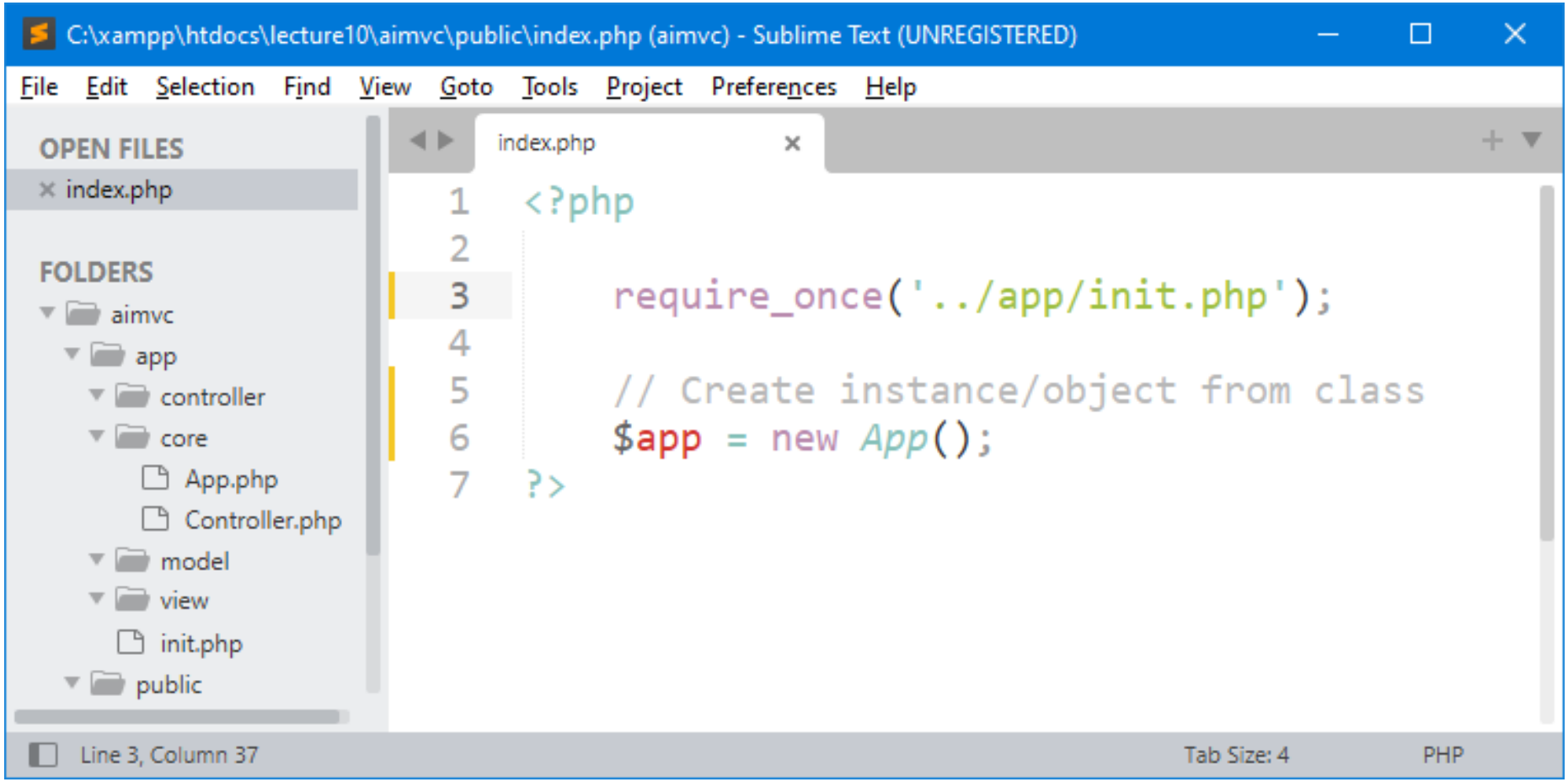


PHP - The ___construct Function:
A constructor allows you to initialize an object's properties upon creation of the object.

PHP - The ___destruct Function:
A destructor is called when the object is destructed or the script is stopped or exited.

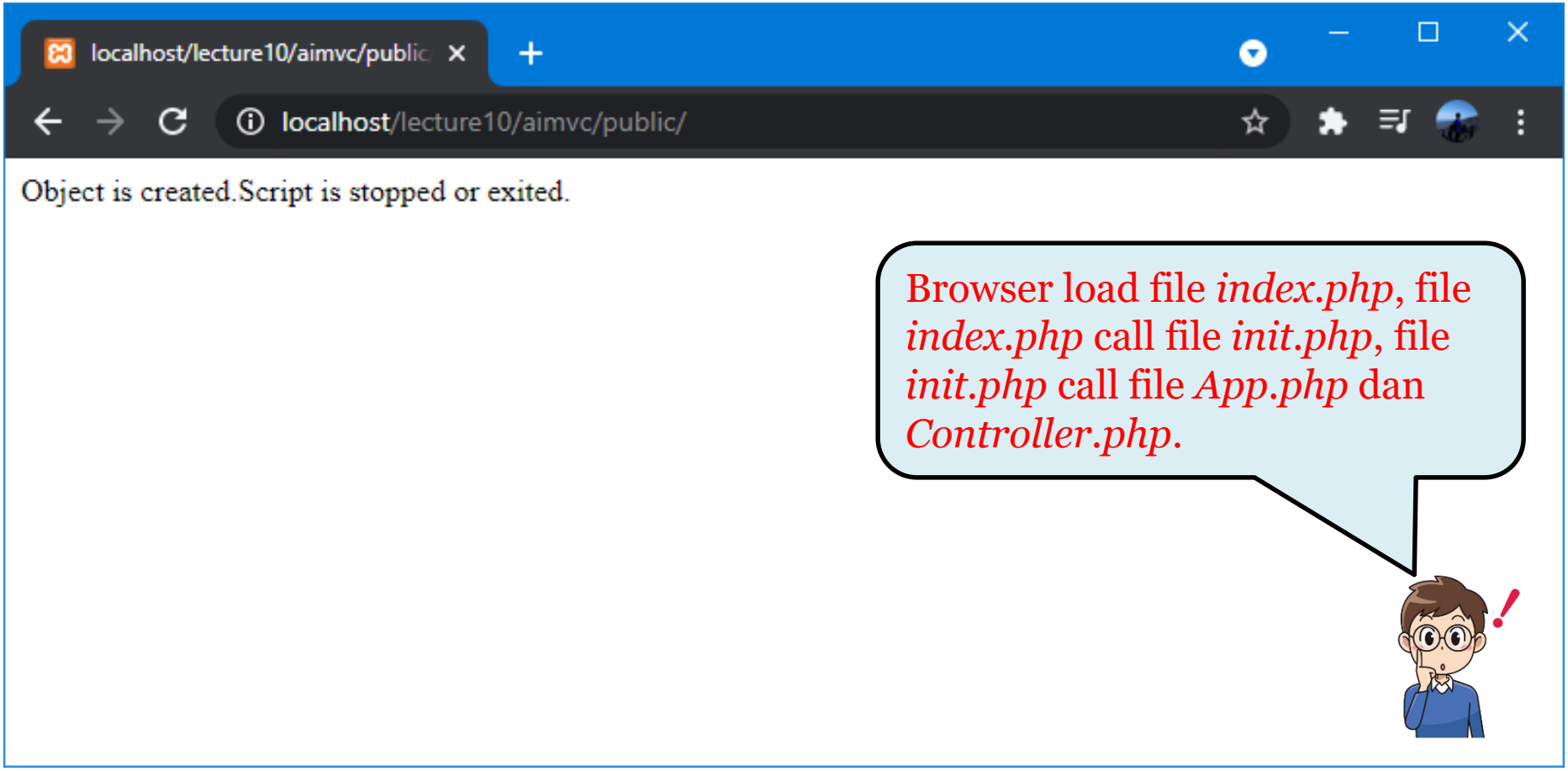
Instansiasi Class (Object Class) App

Instansiasi **class App** (*App.php*) dari file ***index.php*** adalah pembuatan instance/objek dari *class* App, dimana kita akan coba menampilkan/view sesuatu dari *class* tersebut. Perhatikan di dalam *constructor* dari class App sudah terdapat *statement echo* (*which is used to display the output*) dimana akan digunakan untuk menampilkan *output* sederhana.



Test *your* Framework's Structure

Uniform Resource Locator (URL): http://localhost/lecture10/aimvc/public/



Pada saat jalankan (*run*) PHP Files di *browser*, maka *output* yang diharapkan (*giving the expected output*) yaitu seperti diatas.

2 - Penerapan *Routing* (*route*)

[*URL Parsing with Router or URL Routing*]

Istilah/Term Routing

- ***Routing*** adalah proses dimana suatu *item* dapat sampai ke tujuan dari satu lokasi ke lokasi lain. Dalam hal ini, *item* yang dimaksud adalah halaman aplikasi website. Para *developer* dapat menentukan sendiri halaman yang akan muncul pada saat dikunjungi oleh *users*.

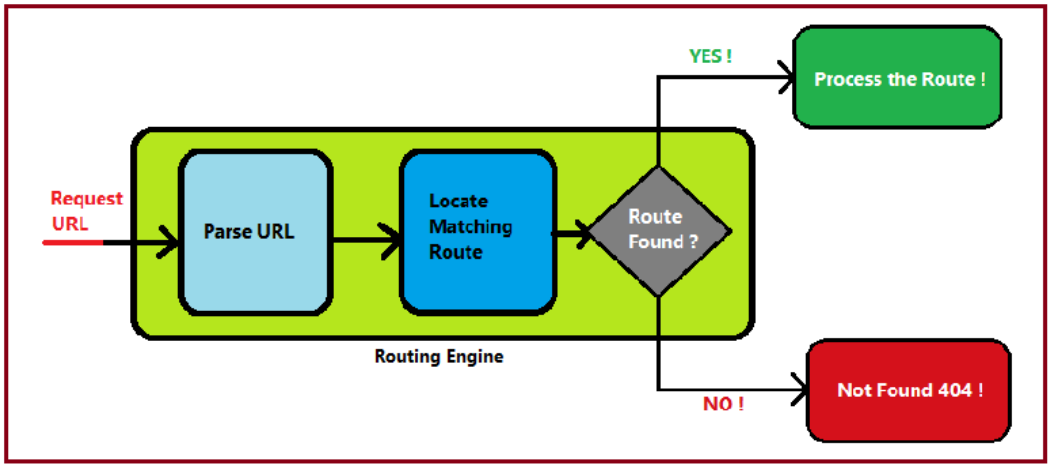
- ***Routing*** is the process of selecting a path for traffic in a network or between or across multiple networks. Broadly, routing is performed in many types of networks, including circuit-switched networks, such as the public switched telephone network (PSTN), and computer networks, such as the Internet. - Wikipedia

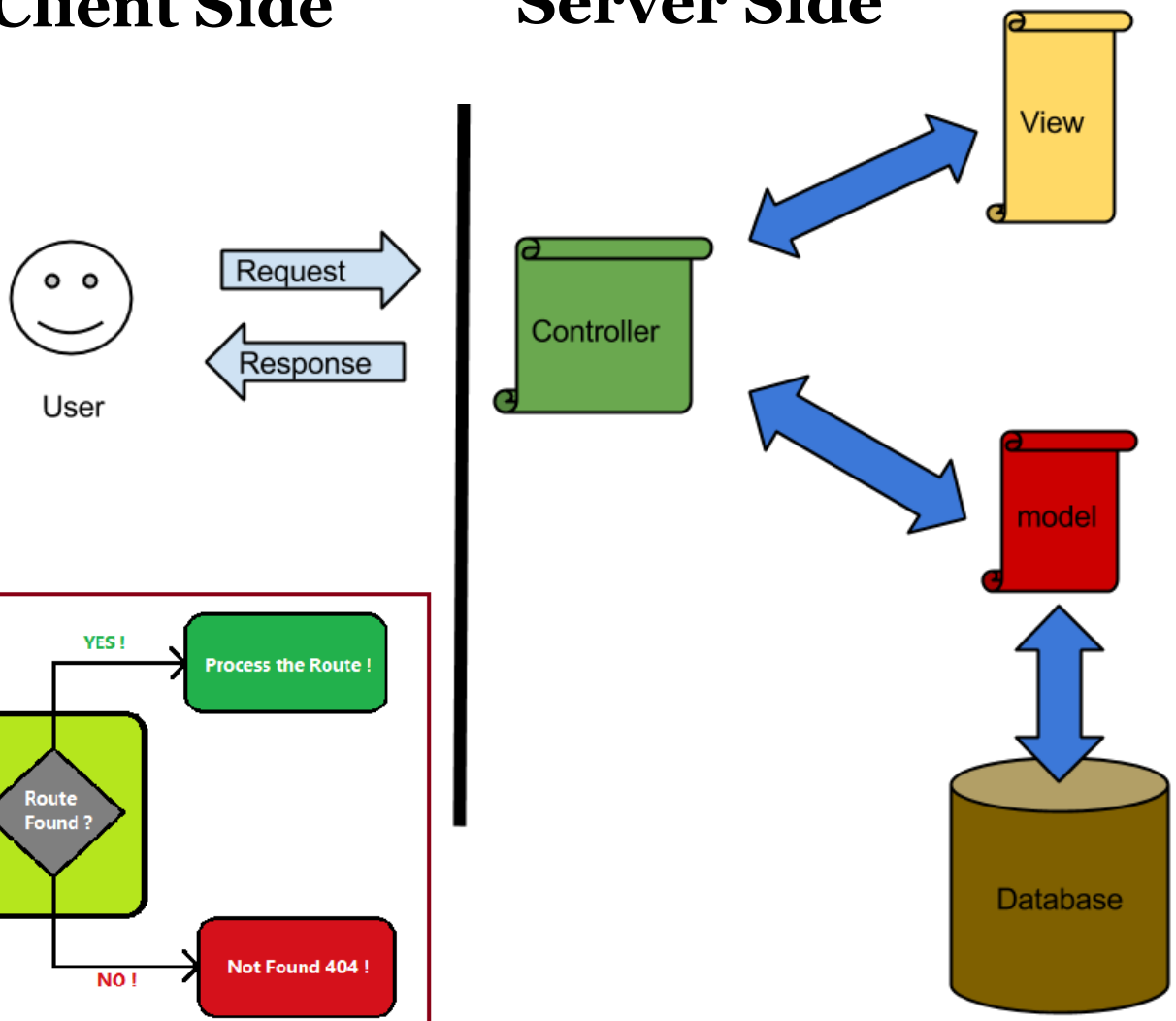
Contoh penggunaan Routing:
Pada saat *user* mengunjungi halaman *dashboard*, maka *kita* dapat menentukan tampilan apa yang akan muncul, apakah itu hanya berupa tulisan (*text*), berupa halaman *controller*, berupa halaman *view*, maupun halaman *error*.

Route dapat menghandle semua perintah yang telah dideklarasikan atau bahkan ***route*** bisa dianalogikan sebagai peta petunjuk bagaimana alur navigasi aplikasi yang sedang dibangun atau juga bisa disebut *Router* merupakan suatu rute/jalan/trayek.

Framework Routing

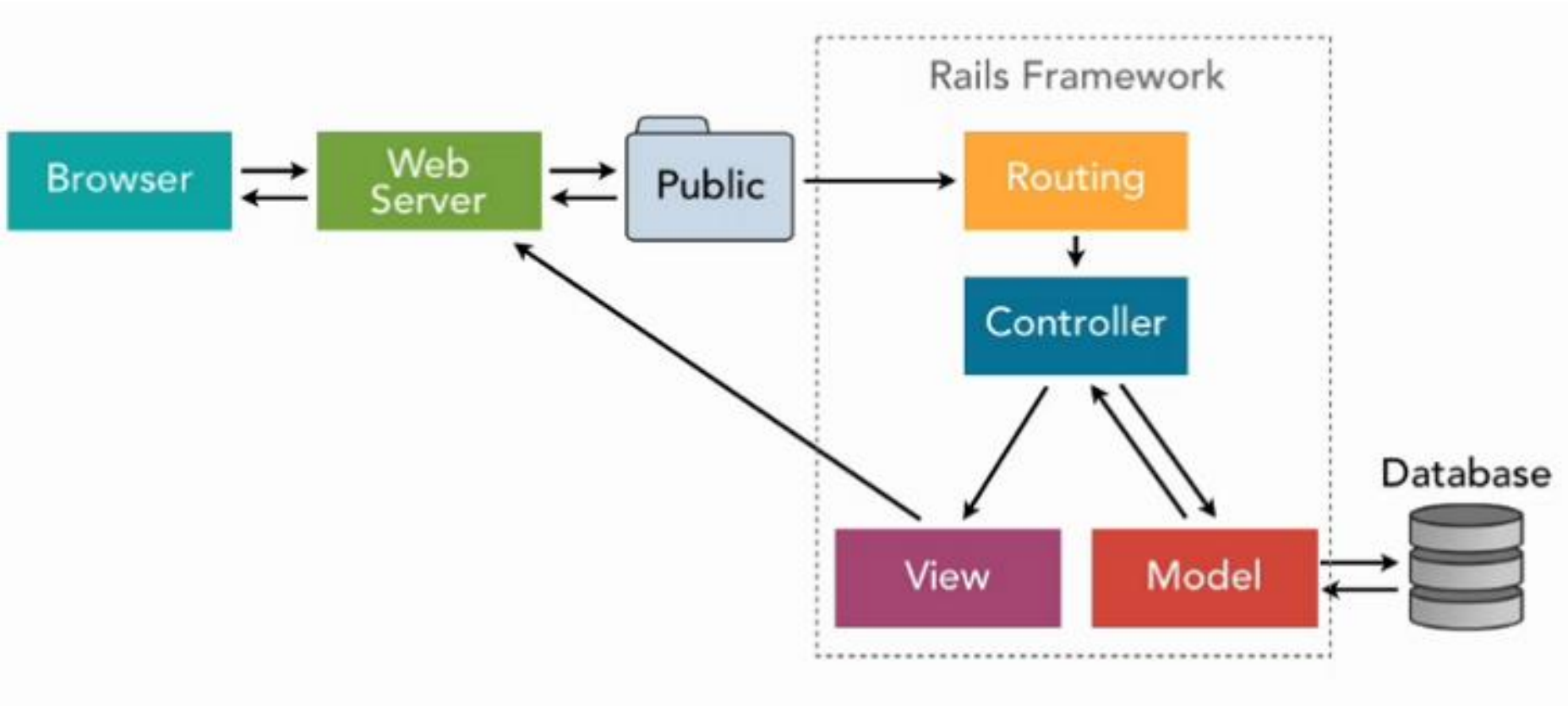
Pada saat *user* mengunjungi halaman *home*, maka kita dapat menentukan tampilan apa yang akan muncul, apakah itu hanya berupa tulisan (*text*), berupa halaman *controller*, berupa halaman *view*, maupun halaman *error*.





Rails Architecture

Ruby on **Rails**, or **Rails**, is a server-side web application framework written in Ruby under the MIT License. Rails is a ***model–view–controller*** (MVC) framework, providing default structures for a database, a web service, and web pages.



URL Routing

Typically there is a one-to-one relationship (*hubungan satu-ke-satu*) between a URL string and its corresponding controller class/method. The segments in a URL (*bagian dalam URL*) normally follow this pattern:

*http:/www.example.com/**class**/**method**/**parameter1**/**parameter2**/*

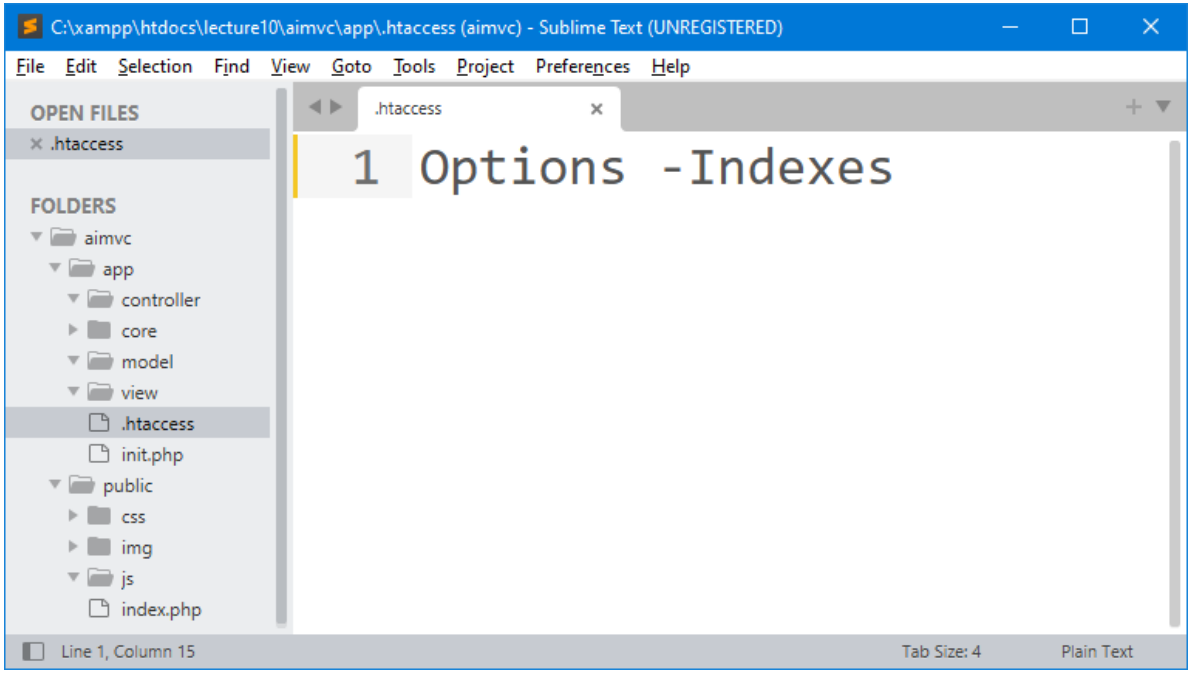
What is a Controller?
A Controller is simply a class file that is named in a way that can be associated with a URI.

Class merupakan *controller class* yang sudah dibuat dan biasanya segmen kedua (***method***) dari URL disiapkan untuk nama *metode*.

Setting *your* own routing rules

1 - Restrict Users Access (*private*)

Block/restrict access permission to ***private*** directory (*app directory*):

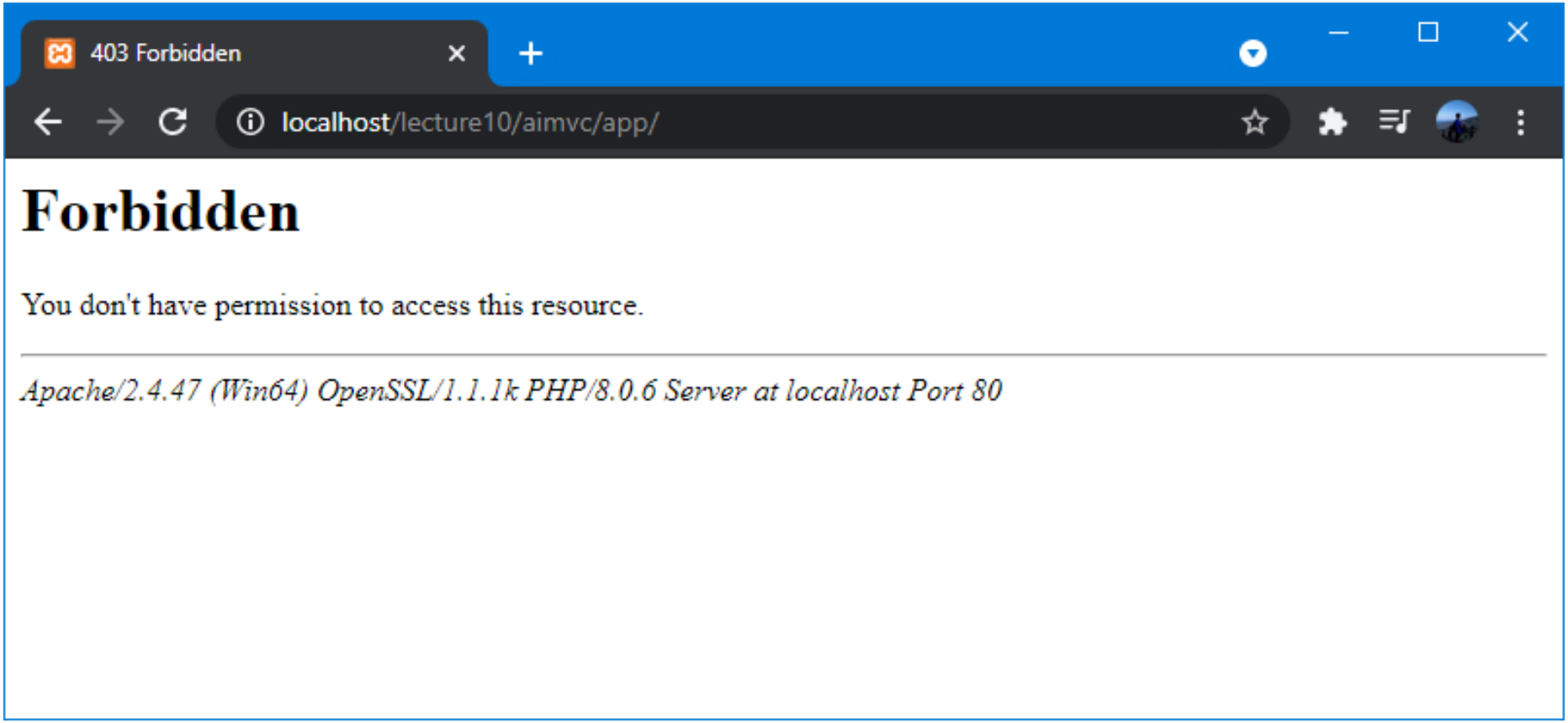


Create file ***.htacces*** di dalam folder app. "***Options –Indexes***" artinya jika ada *users* membuka folder app dan folder-folder yang lain di dalamnya maka jangan tampilkan isi folder-nya, selama di dalam *folder* tersebut tidak ada file *index*. Dalam arti user tidak dapat diberi izin.

File *.htaccess* adalah cara untuk mengkonfigurasi rincian website kita tanpa perlu mengubah *file* konfigurasi *server*.

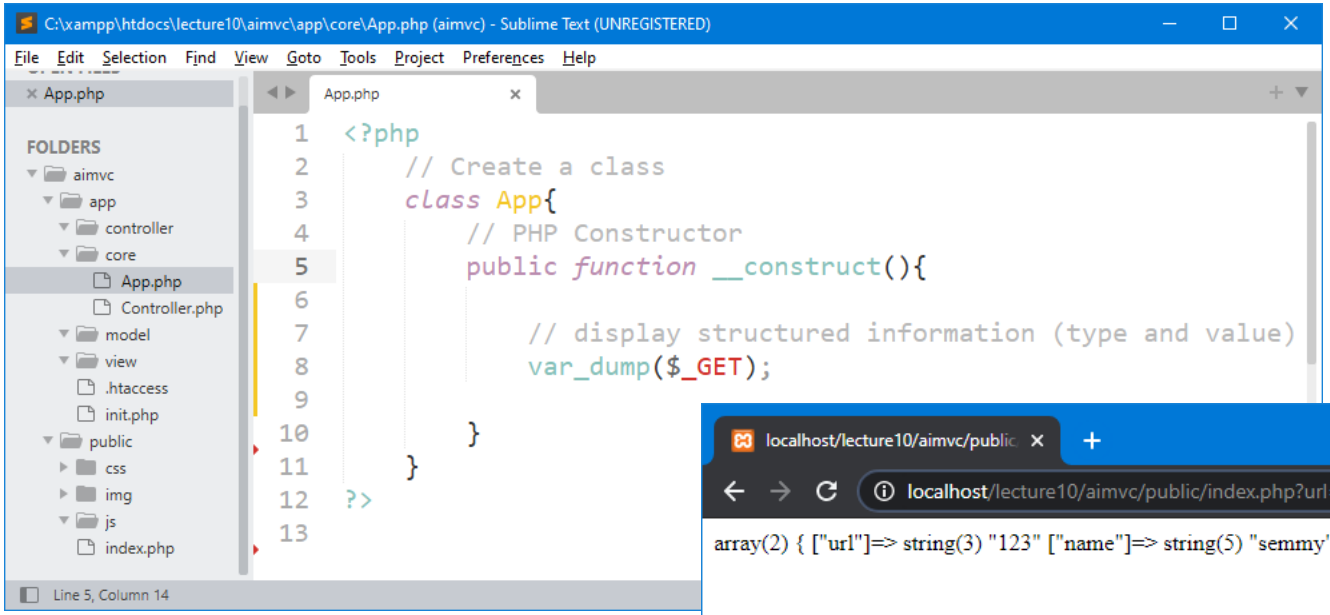
1 - Restrict Users Access (*private*)

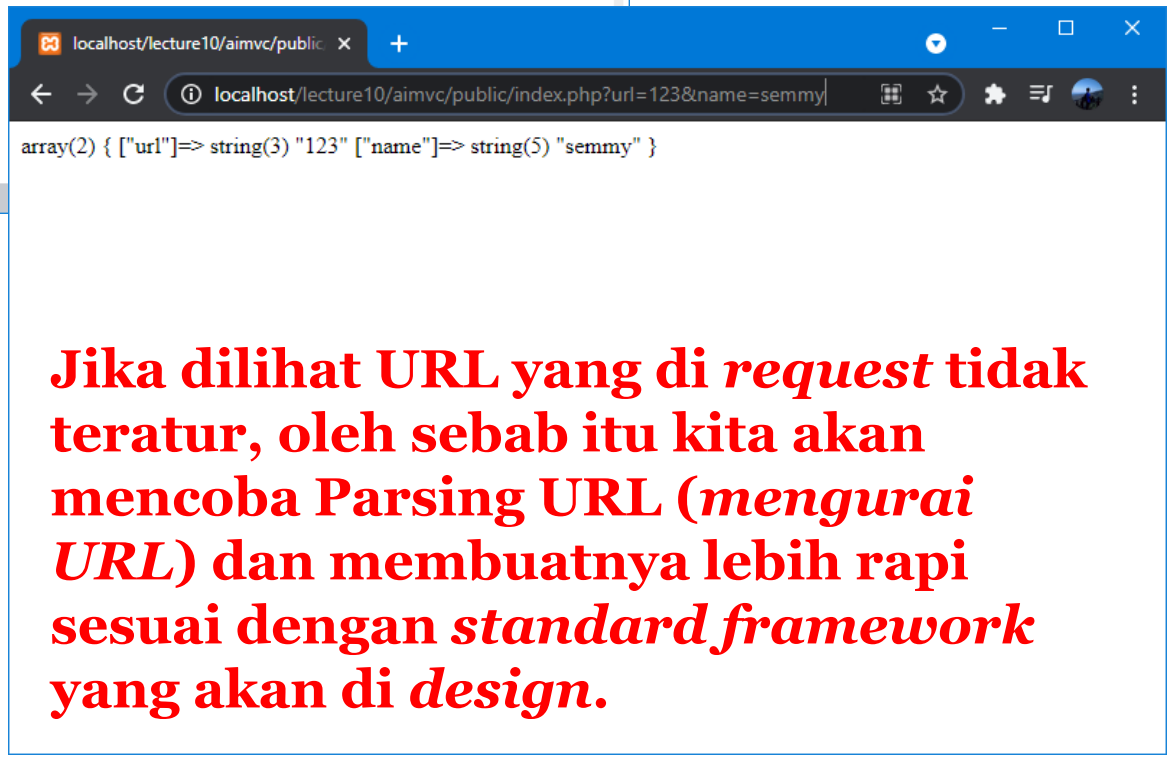
Test apakah *users* masih diberi *access* permission ke ***private*** directory (*app directory*) dan folder-folder yang lain di dalamnya? Jika iya, maka dapat mengaktifkan *.htaccess file* pada *httpd.conf* file.



Test dibuka juga directory http://localhost/lecture10/aimvc/app/view/

2 - Test Request URL Public Directory





Request URL:
http://localhost/lecture10/aimvc/public/index.php?url=123&name=semmy

Jika dilihat URL yang di *request* tidak teratur, oleh sebab itu kita akan mencoba Parsing URL (*mengurai URL*) dan membuatnya lebih rapi sesuai dengan *standard framework* yang akan di *design*.

Apply PHP isset() Function in URL

We used *isset function* to check whether a variable (*get url*) is empty and also check whether the variable (*get url*) is set/declared before setting our own routing rules.

Example isset function:

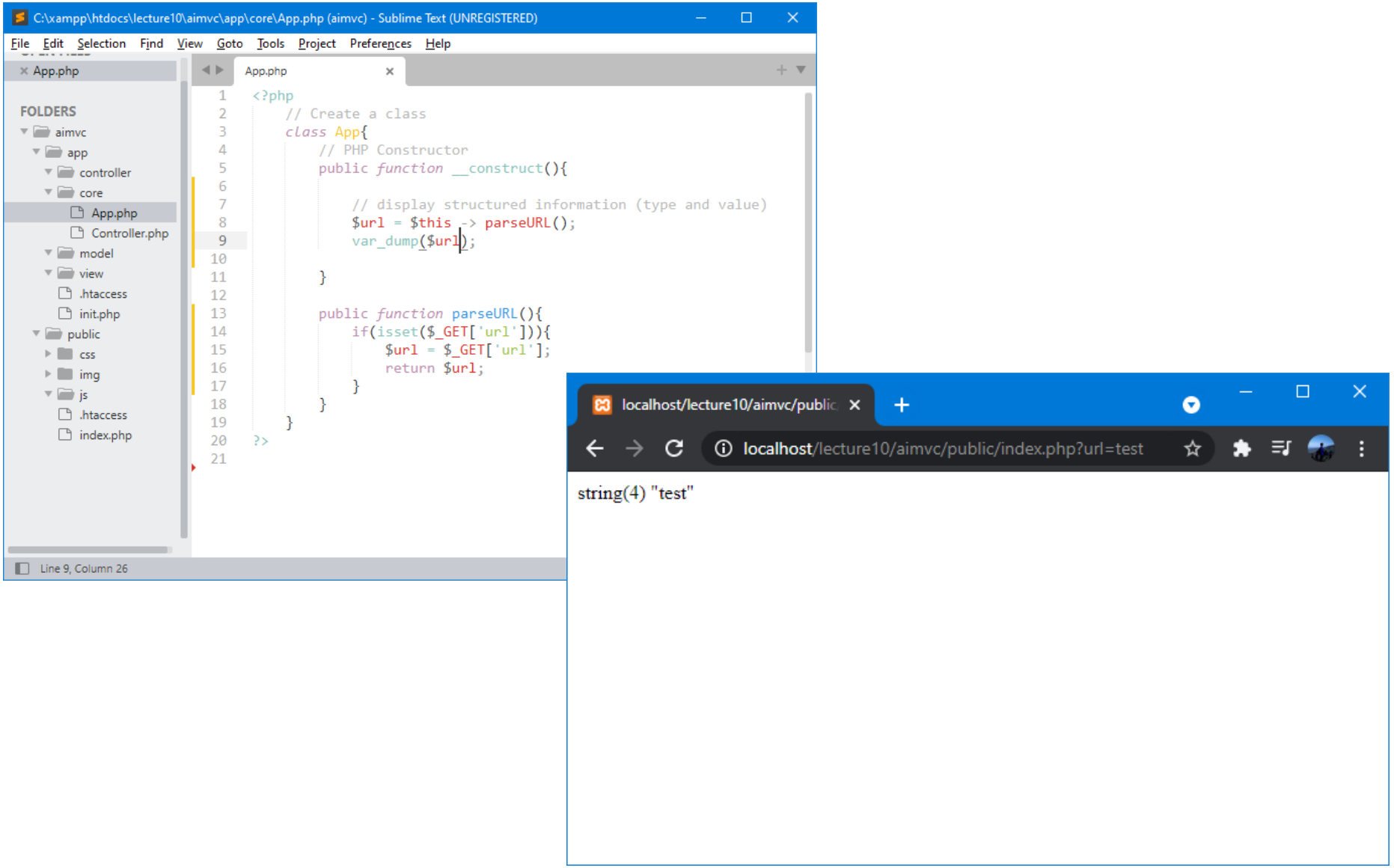
```
<!DOCTYPE html>
<html>
<body>
  <?php
    $a = 0;
    // True because $a is set
    if (isset($a)) {
      echo "Variable 'a' is set.<br>";
    }

    $b = null;
    // False because $b is NULL
    if (isset($b)) {
      echo "Variable 'b' is set.";
    }
  ?>
</body>
</html>
```

Return Value:	TRUE if *variable* exists and is not NULL, FALSE otherwise
Return Type:	Boolean
PHP Version:	4.0+
PHP Changelog:	PHP 5.4: Non-numeric offsets of strings now returns FALSE

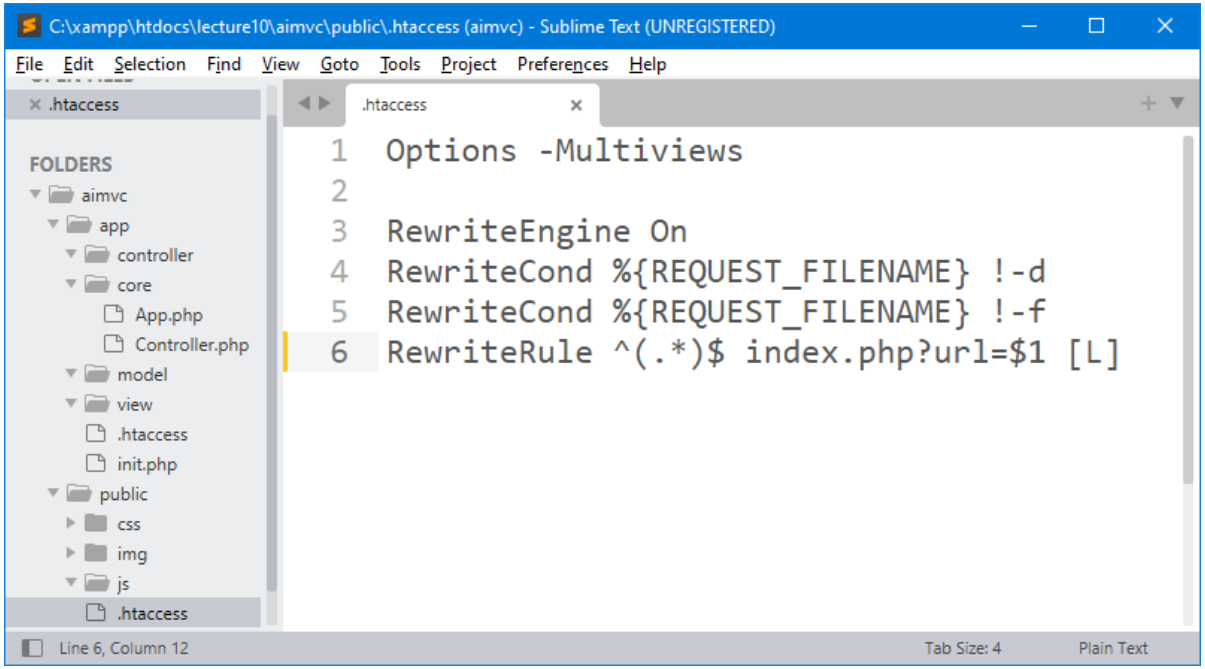
The isset() function checks whether a variable is set, which means that it has to be declared and is not NULL. This function returns true if the variable exists and is not NULL, otherwise it returns false.

3 – Process Parsing URL



4 - Avoid Users Access (*public*)

Menghindari *kesalahan* access *file* atau ***public*** folder (*public directory*).



Create file ***.htacces*** di dalam folder public. "***Options -Multiviews***" artinya kita akan menghindari kesalahan akses file ataupun directory dari URL (*Uniform Resource Locator*).

Karena *users* akan mengakses langsung *file* class bukan ke *folder* yang ada di ***public directory*** melalui URL. Maka kesalahan akses ataupun ***SQL Injection techniques*** (*backend database manipulation to access information*) dapat kita hindari.

4 - Avoid Users Access (*public*)

RewriteCond %{REQUEST_FILENAME} !-d
RewriteCond %{REQUEST_FILENAME} !-f

Rewrite condition (**RewriteCond**) tests along with a corresponding rule (**RewriteRule**) if the prior conditions pass.

Jika URL yang di *request* adalah *directory/folder* (*d*) dan *file* (*f*) maka akan diabaikan (*ignored*). Dalam arti, *request* yang diinginkan hanyalah berupa *method*.

4 - Avoid Users Access (*public*)

RewriteRule ^(.*)$ index.php?url=$1 [L]

RewriteRule defines a particular rule.

Fungsi Regex ^(.*)$ artinya :

1. Symbol caret (^) artinya yang ditulis mulai dari awal url (*match the beginning of a line*) setelah penulisan url "/aimvc/public/".
2. in regular expressions, symbol dot (.) is a special character used to match any one character. Jadi, akan ambil semua character sesudah "aimvc/public".
3. A regular expression followed by an asterisk (*) matches zero or more occurrences of the regular expression. Jada ambil character satu persatu dari url.
4. If a **dollar sign** ($) is at the end of the entire **regular expression**, it matches the end of a line. Jadi akan dicek sampai semua character (*text*) yang ada dalam url selesai.

Fungsi Regex index.php?url=$1 artinya:

Arahkan ke *file* index.php yang mengirimkan url dan disimpan di $1 (*placeholder*).

Flag [L] artinya:

- The [L] flag causes *mod_rewrite* to stop processing the rule set.
- In most contexts, this means that if the rule matches, *no further rules will be processed*.

Request URL without Format

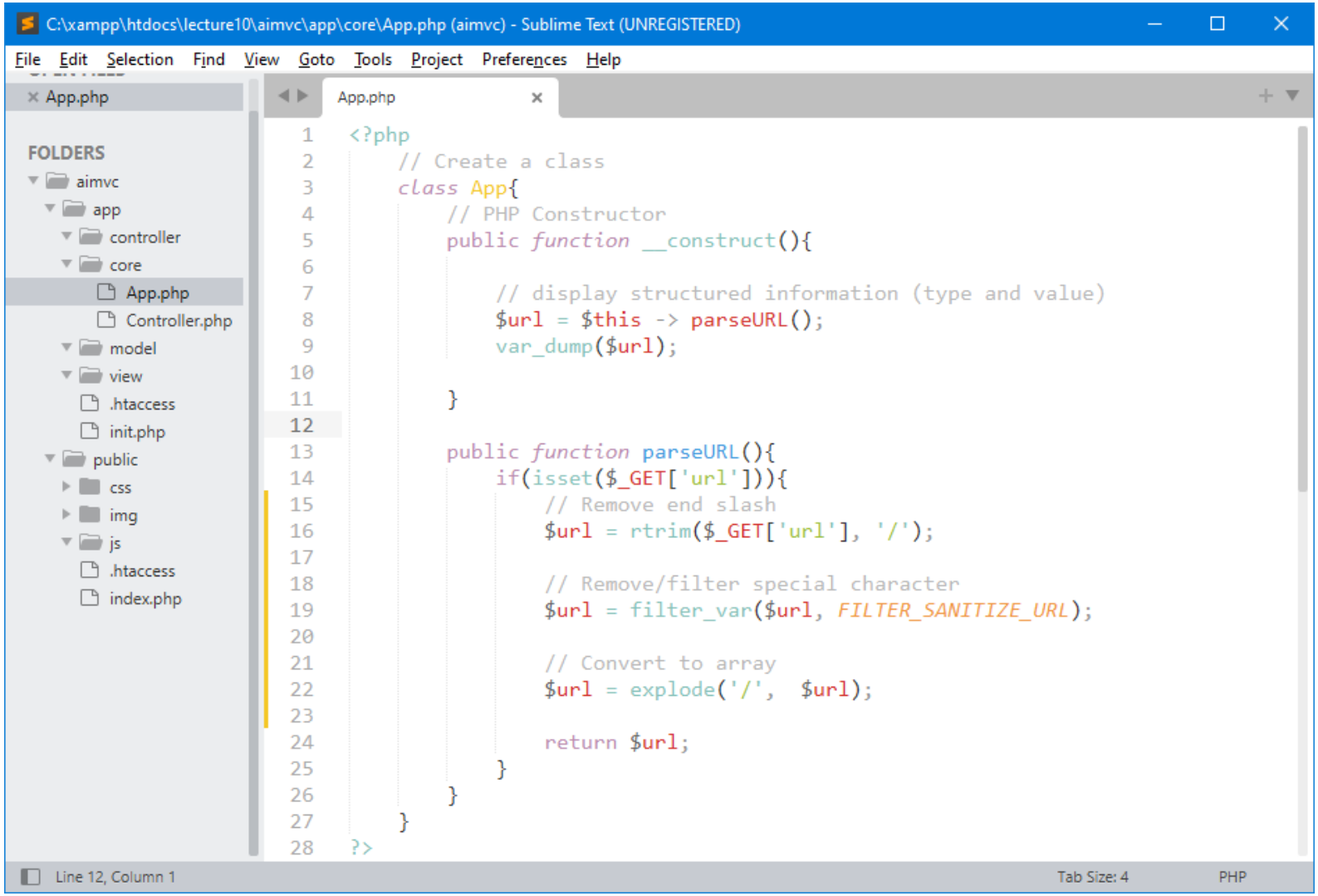
Langkah awal untuk *Request URL*, *parsing URL* dan dapatkan controller class & method dengan memanfaatkan .htaccess *file* success !!!

Setting *your* own routing rules

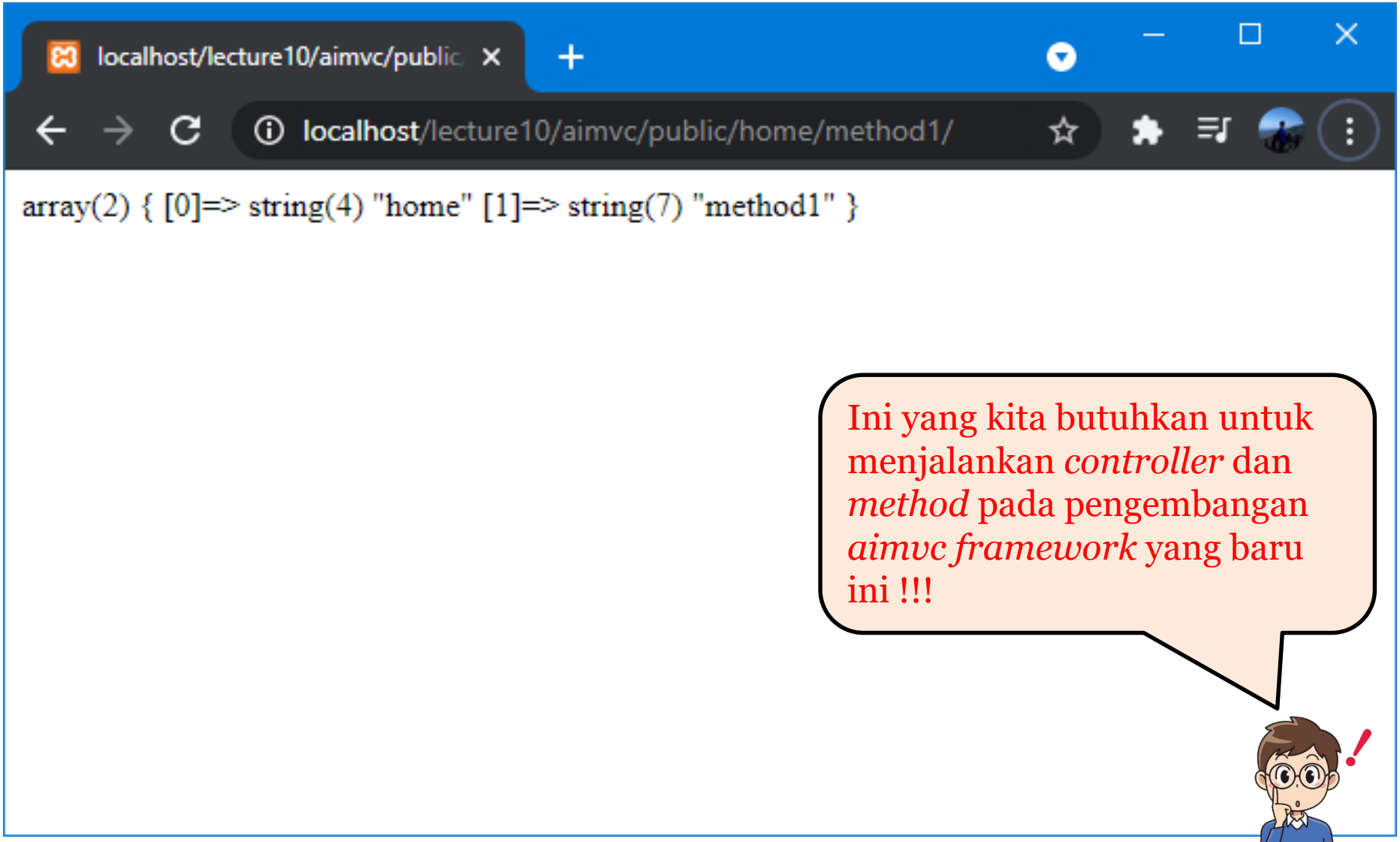
3 – Remove end “/” in Url, Special Character and Covert to Array.

Handling Request URL

Handle more *routing rules* when *users* try to request through URL.



Test Router - Request URL



exercise for students

Latihan untuk *Students*

Create directory : C:\xampp\htdocs\lecture11\
Create file : _.php

Buat semua contoh yang sudah di pelajari sebelumnya.

END PRESENTATION

Thank you for your attention

Instructor: S – W – T

THANK YOU